



Microservices Architecture: When to Use What

A Practical Guide to Choosing the Right Patterns

2.

Agenda



1. **Microservices Concepts and Communication Patterns**
2. **Infrastructure, Integration, and Data Management**
3. **Migration Strategies and Key Takeaways**

3.



Microservices Concepts and Communication Patterns

4.



Why Microservices?

Why move beyond monoliths?

Monoliths can limit agility and slow down scaling.

Unlock scalability and resilience

Microservices scale independently and recover from failures faster.

Enable independent deployments

Teams can release features without waiting for others.

When are microservices the right fit?

Best for complex, evolving systems needing flexibility.

5.



Synchronous vs Asynchronous Calls

REST: Simple, Debuggable Requests

- Best for straightforward request–response scenarios
- Widely supported and easy to troubleshoot

Async Events: Resilience & Scalability

- Enables loose coupling and non-blocking operations
- Supports high throughput and system resilience

gRPC: Fast Internal Communication

- Optimized for low-latency, high-performance service calls
- Ideal for internal microservice interactions

Async + Saga: Long-Running Workflows

- Manages distributed transactions across services
- Ensures eventual consistency for complex processes

6.



REST, gRPC, and Events Compared

REST: External APIs & CRUD Operations

Simple, human-readable, ideal for public interfaces

Events: State Changes & Async Flows

Enables loose coupling and high scalability

gRPC: Fast Internal Communication

Binary protocol, low latency, best for service-to-service

When to Avoid Each Pattern

REST: Not for high-performance internals; gRPC: Not for browsers; Events: Not for strict real-time



Event-Driven Architecture Essentials

Enables Independent Service Evolution

Services can change and deploy without tight coupling

Ideal for High Throughput & Multiple Reactions

Supports scenarios like orders, logs, and payments

Not Suited for Simple CRUD or Low Observability

Avoid if debugging discipline is weak or app is basic

Example: Order Placed Event

Triggers inventory update, payment processing, notifications

8.



Infrastructure, Integration, and Data Management

9.



API Gateway vs BFF

Pattern	Primary Role	Best Use Case
API Gateway	Single entry, auth, routing	Cross-cutting concerns
BFF	Tailored APIs for clients	Client-specific needs



Service Discovery and Databases



Service Discovery Strategies

- Kubernetes offers built-in DNS-based discovery.
- Consul or Eureka are preferred for non-Kubernetes setups.



Database Ownership in Microservices

- Each service should manage its own database for true autonomy.
- Shared databases are only acceptable during legacy migrations.



Golden Rule

- If two services share a database, they are not true microservices.

11.



Saga, Choreography, and Orchestration



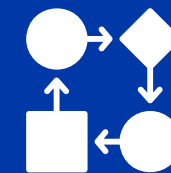
Saga Pattern: Workflow Management

Coordinates multi-service transactions with eventual consistency



Choreography: Decentralized Simplicity

Services react to events for simple, scalable flows



Orchestration: Centralized Control

A central coordinator manages complex, visible workflows

12.



CQRS and Event Sourcing



CQRS: Split Read and Write Operations

- Boosts performance and manages complexity
- Ideal for systems with heavy read or complex business logic



Event Sourcing: Store State as Events

- Provides a complete audit trail of all changes
- Enables rebuilding system state from event history



Best Fit Use Cases

- Reporting, dashboards, and compliance-driven systems



Choosing a Message Broker

Message Broker	Best Use Case
RabbitMQ	Task queues, workflows
Kafka	Event streaming, analytics
Redis Streams	Lightweight eventing

14.



Migration Strategies and Key Takeaways

15.



Migration Patterns and Final Tips



Strangler Fig Pattern for Migration

Incrementally replace monolith features with microservices



Anti-Corruption Layer for Legacy Systems

Isolate new services from legacy complexities



Let Real Problems Guide Your Choices

Adopt patterns based on actual needs, not industry trends

16.

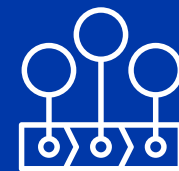


Key Takeaways



Begin with Simplicity

Adopt basic patterns and scale as needs grow



Match Patterns to Problems

Select communication and data strategies for your context



Use Microservices Judiciously

Treat microservices as tools, not mandatory rules